Generic Programming

Advanced Programming

Brent van Bladel

Introduction

“Generic programming centers around the idea of abstracting from concrete algorithms to obtain generic algorithms that can be combined with different data representations to produce a wide variety of useful software.”

— Musser, David R. & Stepanov, Alexander A.

Introduction

"Generic programming centers around the idea of abstracting from concrete algorithms to obtain generic algorithms that can be combined with different data representations to produce a wide variety of useful software."

— Musser, David R. & Stepanov, Alexander A.

"Lift algorithms and data structures from concrete examples to their most general and abstract form."

— Bjarne Stroustrup

Introduction

```
1.  int sum(int a, int b){
2.      return a + b;
3.  }
4.
5.  double sum(double a, double b){
6.      return a + b;
7.  }
8.
9.  long sum(long a, long b){
10.     return a + b;
11. }
```

“concrete algorithms”

Introduction

```
1.  int sum(int a, int b){
2.      return a + b;
3.  }
4.
5.  double sum(double a, double b){
6.      return a + b;
7.  }
8.
9.  long sum(long a, long b){
10.     return a + b;
11. }
```

"concrete algorithms"

Replace

```
1.  Number sum(Number a, Number b){
2.      return a + b;
3.  }
```

"generic algorithms"

Templates

```
1.  int sum(int a, int b){
2.      return a + b;
3.  }
4.
5.  double sum(double a, double b){
6.      return a + b;
7.  }
8.
9.  long sum(long a, long b){
10.     return a + b;
11. }
```

"concrete algorithms"

Generate*

```
1.  template <typename Number>
2.  Number sum(Number a, Number b){
3.      return a + b;
4.  }
```

"generic algorithms"

*at compile-time

Templates

```
1.  template <typename Container>
2.  void printContainer(Container& container) {
3.    for (auto element : container)
4.        cout << element << endl;
5.  }
6.
7.  int main() {
8.     vector<int> v{1,2,3,4,5};
9.     printContainer(v);
10. }
```

"generic algorithms"

Function Templates

```
1.  template <typename Number>
2.  Number sum(Number a, Number b){
3.      return a + b;
4.  }
```

```
1.  template <typename T1, typename T2>
2.  void print(T1 a, T2 b){
3.      cout << a << " " b << endl;
4.  }
```

Function Templates

```
1.  template <typename Number>
2.  Number sum(Number a, Number b){
3.      return a + b;
4.  }
```

```
1.  template <typename T1, typename T2>
2.  void print(T1 a, T2 b){
3.      cout << a << " " b << endl;
4.  }
```

```
1.  cout << sum(5, 8) << endl;
2.  cout << sum(5.0, 8.5) << endl;
3.
4.  print(5, 8);
5.  print(5.0, "Hello");
```

Template Instantiation

```
1.  template <typename Number>
2.  Number sum(Number a, Number b){
3.      return a + b;
4.  }
```

The compiler generates no binary code from a source file with only template definitions.

→ Templates must be instantiated in order for code to be generated!

Template Instantiation

```
1.  template <typename Number>
2.  Number sum(Number a, Number b){
3.      return a + b;
4.  }
```

Implicit Template Instantiation:
When calling a template function, it is implicitly instantiated with the called type.
⟶ Code is generated for this function.

```
1.  cout << sum(5, 8) << endl;
2.  cout << sum(5.0, 8.5) << endl;
```

<u>Instantiates and generates</u>:
int sum(int, int)
double sum(double, double)

Template Instantiation

```
1.  template <typename Number>
2.  Number sum(Number a, Number b){
3.      return a + b;
4.  }
```

Explicit Template Instantiation:
An explicit instantiation definition forces instantiation of the function.
⟶ Code is generated for this function.

```
1.  template int sum(int, int);
2.  template double sum(double, double);
```

<u>Instantiates and generates</u>:
int sum(int, int)
double sum(double, double)

Template Argument Deduction

```
1.  template <typename Number>
2.  Number sum(Number a, Number b){
3.      return a + b;
4.  }
```

```
1.  // compilation error:
2.  // no matching function for call to 'sum(int, double)'
3.  cout << sum(5, 8.5) << endl;
```

The template type must be deducible at compile-time based on the argument types.

→ **If not possible: compiler error!**

Template Argument Deduction

```
1.  template <typename Number>
2.  Number sum(Number a, Number b){
3.      return a + b;
4.  }
```

```
1.  // compilation error:
2.  // no matching function for call to 'sum(int, double)'
3.  cout << sum(5, 8.5) << endl;
4.
5.  // implicit conversion from int to double
6.  cout << sum<double>(5, 8.5) << endl;
```

The template type must be deducible at compile-time based on the argument types.

→ **If not possible: compiler error!**

You can bypass template argument deduction by providing the template type explicitly.

Template Argument Deduction

```
1.  template int sum<int>(int, int); // explicit template type
2.  template int sum<>(int, int); // template type deduced
3.  template int sum(int, int); // template type deduced
4.
5.  int main() {
6.    cout << sum<int>(5, 8) << endl; // explicit template type
7.    cout << sum<>(5, 8) << endl;    // template type deduced
8.    cout << sum(5, 8) << endl;      // template type deduced
9.  }
```

You can bypass template argument deduction by providing the template type explicitly.

→ **Both in implicit and explicit instantiations!**

Explicit Specialization of a Template Function

```
1.  template <typename Number>
2.  Number sum(Number a, Number b){
3.    return a + b;
4.  }
5.
6.  template <>
7.  double sum<double>(double a, double b){
8.    int rounded = a + b;
9.    return rounded ;
10. }
11.
12. int main() {
13.   cout << sum(5, 8) << endl;      // generic version
14.   cout << sum(5.0, 8.5) << endl; // specific version
15. }
```

You can overload the general version of the template function with a specific implementation.

Example:
vector<TElement> vs vector<bool>

If constexpr

```
1.  template<typename Number>
2.  void sum(Number a, Number b) {
3.     if constexpr (std::is_same<Number,double>::value) {
4.          int rounded = a + b;
5.          return rounded ;
6.     } else {
7.          return a + b;
8.  }}
```

Evaluate if statement at compile-time.

→ Generated functions will not contain the if.

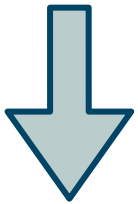
Template Function Overloading

```
1.  template <typename Number>
2.  Number sum(Number a, Number b){
3.    return a + b;
4.  }
5.
6.  template <typename Number>
7.  Number sum(Number a, Number b, Number c){
8.    return a + b + c;
9.  }
10.
11. int main() {
12.   cout << sum(5, 8) << endl;
13.   cout << sum(5, 8, 2) << endl;
14. }
```

Template functions can be overloaded similar to non-template functions.

Abbreviated Template Function (since c++20)

```
1.  void print(auto a, auto b) {
2.      cout << a << " " b << endl;
3.  }
```



```
1.  template <typename T1, typename T2>
2.  void print(T1 a, T2 b) {
3.     cout << a << " " b << endl;
4.  }
```

Template functions can be abbreviated with auto parameters.

→ Translated to template function by compiler.

auto keyword (since c++11)

The **auto** keyword allows for automatic type-deduction at compile-time.

→ **Benefits?**

```
1.  auto x = 42;
```

auto keyword (since c++11)

The **auto** keyword allows for automatic type-deduction at compile-time.

```
1.    auto x = 42;
```

→ **Benefits:**

- Correctness: auto guarantees the correct type is used.

auto keyword (since c++11)

The **auto** keyword allows for automatic type-deduction at compile-time.

```
1.    auto x = 42;
```

→ **Benefits:**

- Correctness: auto guarantees the correct type is used.
- Maintainability and robustness: When an expression's type changes, auto will continue to resolve correctly.

auto keyword (since c++11)

The **auto** keyword allows for automatic type-deduction at compile-time.

```
1.    auto x = 42;
```

→ **Benefits:**

- Correctness: auto guarantees the correct type is used.
- Maintainability and robustness: When an expression's type changes, auto will continue to resolve correctly.
- Performance: auto guarantees no implicit conversions.

auto keyword (since c++11)

The **auto** keyword allows for automatic type-deduction at compile-time.

```
1.    auto x = 42;
```

→ **Benefits:**

- Correctness: auto guarantees the correct type is used.
- Maintainability and robustness: When an expression's type changes, auto will continue to resolve correctly.
- Performance: auto guarantees no implicit conversions.
- Usability: easier to use for long or complex types (such as function pointers)

auto keyword (since c++11)

"write code against interfaces, not implementations"

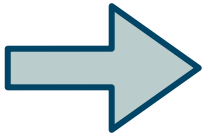
```
1.  set<string> getData() {
2.      return {"one", "two", "three"};
3.  }
```

```
1.  vector<int> getData() {
2.      return {1, 2, 3};
3.  }
```

```
1.  int main() {
2.      auto data = getData();
3.      for (auto element : data)
4.          cout << element << endl;
5.  }
```

Trailing Return Type (since c++11)

```
1.  vector<int> getData() {
2.      return {1, 2, 3};
3.  }
```



```
1.  auto getData() -> vector<int> {
2.      return {1, 2, 3};
3.  }
```

Trailing Return Type (since c++11)

- Can use parameters as part of the return type:

```
1.  template <typename Number>
2.  auto sum(Number a, Number b) -> decltype(a + b) {
3.    return a + b;
4.  }
```

Trailing Return Type (since c++11)

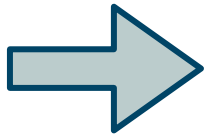
- Can use parameters as part of the return type.
- Can access class (private) type aliases without namespace specifier:

```
1.  class Rectangle {
2.    private:
3.        using Corners = vector<Coordinate>;
4.        Corners _corners;
5.    public:
6.        auto getCorners() -> Corners;
7.  };
8.
9.  auto Rectangle::getCorners() -> Corners {
10.   return _corners;
11. };
```

Trailing Return Type (since c++11)

- Can use parameters as part of the return type.
- Can access class type aliases without namespace specifier.
- Easy to switch to automatic return type deduction in the future*:

```
1.  auto getData() -> vector<int> {
2.      return {1, 2, 3};
3.  }
```

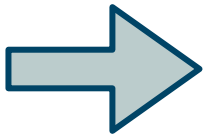


```
1.  auto getData() {
2.      return vector<int>{1, 2, 3};
3.  }
```

*automatic return type deduction (since c++14)

Automatic Return Type Deduction (since c++14)

```
1.  auto getData() -> vector<int> {
2.      return {1, 2, 3};
3.  }
```



```
1.  auto getData() {
2.      return vector<int>{1, 2, 3};
3.  }
```

Function return types can be deduced at compile-time.

Class Templates

```
1.  template <typename Element>
2.  class Queue {
3.      private:
4.          vector<Element> _elements;
5.      public:
6.          void push(const Element& e);
7.          Element front();
8.  };
```

Class Templates

```
1.  template <typename Element>
2.  class Queue {
3.    private:
4.        vector<Element> _elements;
5.    public:
6.        void push(const Element& e);
7.        Element front();
8.  };
```

```
1.  template <typename Element>
2.  void Queue<Element>::push(const Element& e) {
3.    _elements.push_back(e);
4.  }
```

Class Templates

```
1.  template <typename Element>
2.  class Queue {
3.    private:
4.      vector<Element> _elements;
5.    public:
6.      void push(const Element& e);
7.      Element front();
8.  };
```

```
1.  template <typename Element>
2.  void Queue<Element>::push(const Element& e) {
3.    _elements.push_back(e);
4.  }
```

The compiler generates no binary code from a source file with only template definitions.

→ Templates must be instantiated in order for code to be generated!

Class Templates

```
1.  template <typename Element>
2.  class Queue {
3.    private:
4.        vector<Element> _elements;
5.    public:
6.        void push(const Element& e);
7.        Element front();
8.  };
```

```
1.  int main() {
2.      Queue<int> q;
3.      q.push(1);
4.      q.push(2);
5.      q.push(3);
6.      cout << q.front() << endl;
7.  }
```

The compiler generates no binary code from a source file with only template definitions.

⟶ Templates must be instantiated in order for code to be generated!

Class Templates

```
1.  template<typename FirstType, SecondType>
2.  class MyPair {
3.      FirstType _first;
4.      SecondType _second;
5.    public:
6.      MyPair(FirstType a, SecondType b) : _first{a}, _second{b} {};
7.
8.      inline void print() const {
9.          cout << _first << ", " << _second << endl;
10.     }
11. };
```

```
1.  MyPair<int, string> p{5, "Hello"};
2.  p.print();
```

Class Template Argument Deduction (since C++17)

```
1.  template<typename FirstType, SecondType>
2.  class MyPair {
3.      FirstType _first;
4.      SecondType _second;
5.    public:
6.      MyPair(FirstType a, SecondType b) : _first{a}, _second{b} {};
7.
8.      inline void print() const {
9.          cout << _first << ", " << _second << endl;
10.     }
11. };
```

```
1.  MyPair p{5, "Hello"};
2.  p.print();
```

Default Template Arguments

```
1.  template<typename FirstType = int, SecondType = int>
2.  class MyPair {
3.      FirstType _first;
4.      SecondType _second;
5.    public:
6.      MyPair(FirstType a, SecondType b) : _first{a}, _second{b} {};
7.
8.      inline void print() const {
9.          cout << _first << ", " << _second << endl;
10.     }
11. };
```

```
1.  short x = 5, y = 9;
2.  MyPair p{x, y}; // implicit conversion short -> int
3.  p.print();
```

Templates & Inheritance

```
1.  class IReporter {
2.    public:
3.      // compilation error
4.      template<typename StringType>
5.      virtual void print(StringType message) = 0;
6.  };
```

Template functions cannot be virtual.

Templates & Inheritance

```
1.  class IReporter {
2.    public:
3.      // compilation error
4.      template<typename StringType>
5.      virtual void print(StringType message) = 0;
6.  };
```

Template functions cannot be virtual.

→ Templated functions provide compile-time polymorphism, virtual functions provide runtime polymorphism.

Templates & Inheritance

```
1.  class IReporter {
2.    public:
3.      // compilation error
4.      template<typename StringType>
5.      virtual void print(StringType message) = 0;
6.  };
```

Template functions cannot be virtual.

→ Templated functions provide compile-time polymorphism, virtual functions provide runtime polymorphism.

(Size of the vtable would not be known until the entire program has been translated.)

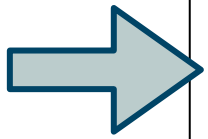
Templates & Inheritance

```
1.  template<typename StringType>
2.  class IReporter {
3.    public:
4.      virtual void print(StringType message) = 0;
5.  };
6.
7.  template<typename StringType>
8.  class Reporter : public IReporter<StringType>{
9.    public:
10.     void print(StringType message) override;
11. };
```

A template class can contain virtual functions!

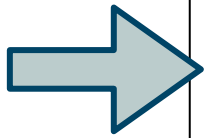
Templates & Inheritance

```
1.  class IReporter {
2.    public:
3.      // compilation error
4.      template<typename StringType>
5.      virtual void print(StringType message) = 0;
6.  };
```



One class with unknown amount of overloaded print functions.

```
1.  template<typename StringType>
2.  class IReporter {
3.    public:
4.      virtual void print(StringType message) = 0;
5.  };
```



Unknown amount of classes with one print function each.

Universal Reference

```
1.  template <typename T>
2.  void func(T && t);
```

A **universal reference** is an rvalue reference to a template parameter, which can then be deduced as either a rvalue or an lvalue reference.

Universal Reference

```
1.  template <typename T>
2.  void func(T && t);
```

A **universal reference** is an rvalue reference to a template parameter, which can then be deduced as either a rvalue or an lvalue reference.

→ Also called a **forwarding reference**.

std::forward (since C++14)

```
1.  struct A {
2.   A(int&& n) { cout << "rvalue A"; }
3.   A(int& n)   { cout << "lvalue A"; }
4.  };
5.
6.  template <typename Number>
7.  A AFactory(Number&& n){
8.    return A{std::forward<Number>(n)};
9.  }
10.
11. int main() {
12.   int x = 1;
13.   AFactory(1); // rvalue A
14.   AFactory(x); // lvalue A
15. }
```

std::forward preserves the value category of its argument.

std::forward (since C++14)

```
1.  struct A {
2.   A(int&& n) { cout << "rvalue A"; }
3.   A(int& n)   { cout << "lvalue A"; }
4.  };
5.
6.  template <typename Number>
7.  A AFactory(Number&& n){
8.    return A{std::forward<Number>(n)};
9.  }
10.
11. int main() {
12.   int x = 1;
13.   AFactory(1); // rvalue A
14.   AFactory(x); // lvalue A
15. }
```

std::forward preserves the value category of its argument.

perfect forwarding: objects passed as lvalue are copied, and objects passed as rvalue are moved.

std::forward (since C++14)

```
1.  struct A {
2.   A(int&& n) { cout << "rvalue A"; }
3.   A(int& n)   { cout << "lvalue A"; }
4.  };
5.
6.  template <typename Number>
7.  A AFactory(Number&& n){
8.    return A{std::forward<Number>(n)};
9.  }
10.
11. int main() {
12.   int x = 1;
13.   AFactory(1); // rvalue A
14.   AFactory(x); // lvalue A
15. }
```

std::forward preserves the value category of its argument.

perfect forwarding: objects passed as lvalue are copied, and objects passed as rvalue are moved.

⟶ avoids excessive copying, and avoids having to write multiple overloads for lvalue and rvalue references.

Parameter pack (since C++11)

A template **parameter pack** is a template parameter that accepts zero or more template arguments.

Syntax: template <typename… PackName>

A function **parameter pack** is a function parameter that accepts zero or more function arguments.

Syntax: functionName(PackName… parameterName)

Parameter pack (since C++11)

```
1.  template <typename Arg, typename...Args>
2.  void print(Arg&& arg, Args&&... args){
3.     cout << arg;
4.     ((cout << ' ' << args), ...);
5.  }
6.
7.  int main() {
8.     print("Grade: ", 15,5, '/', 20);
9.  }
```

Parameter pack (since C++11)

```
1.  template <typename Arg, typename...Args>
2.  void print(Arg&& arg, Args&&... args){
3.      cout << arg;
4.      ((cout << ' ' << args), ...);
5.  }
6.
7.  int main() {
8.      print("Grade: ", 15.5, '/', 20);
9.  }
```

template parameter pack

(0 or more types)

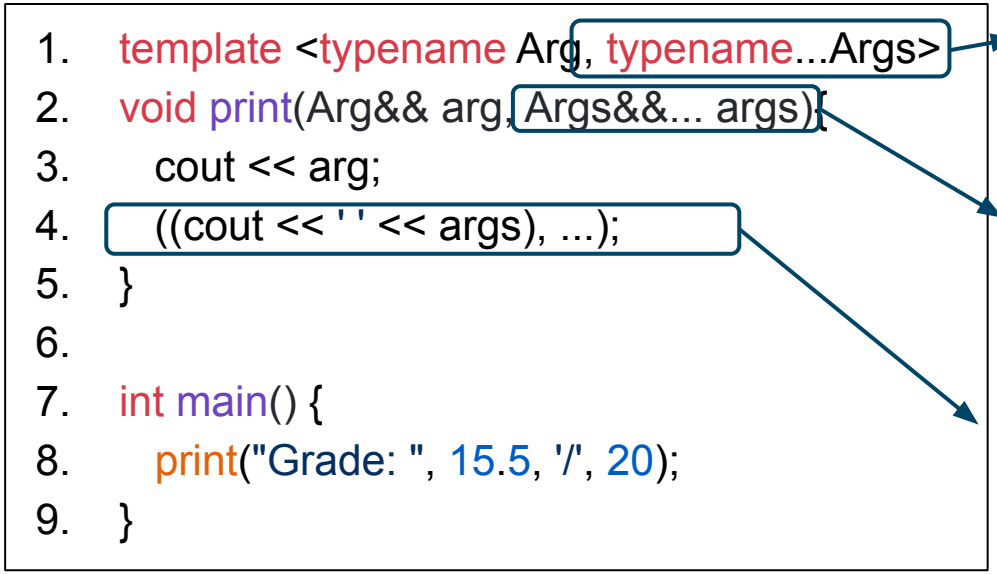
Parameter pack (since C++11)

```
1.  template <typename Arg, typename...Args>
2.  void print(Arg&& arg, Args&&... args){
3.      cout << arg;
4.      ((cout << ' ' << args), ...);
5.  }
6.
7.  int main() {
8.      print("Grade: ", 15.5, '/', 20);
9.  }
```

template parameter pack
(0 or more types)

function parameter pack
(0 or more parameters)

Parameter pack (since C++11)



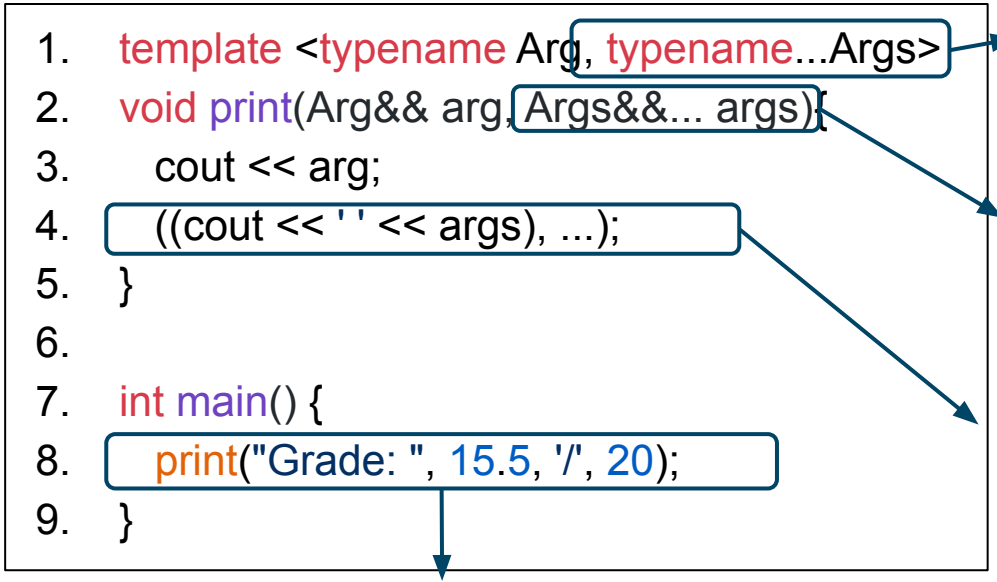
```
1.  template <typename Arg, typename...Args>
2.  void print(Arg&& arg, Args&&... args){
3.     cout << arg;
4.     ((cout << ' ' << args), ...);
5.  }
6.
7.  int main() {
8.     print("Grade: ", 15.5, '/', 20);
9.  }
```

template parameter pack
(0 or more types)

function parameter pack
(0 or more parameters)

fold expression: Reduces a parameter pack over an operator. (since C++17)

Parameter pack (since C++11)



template parameter pack
(0 or more types)

function parameter pack
(0 or more parameters)

fold expression: Reduces a parameter pack over an operator. (since C++17)

Function can be called with any number of parameters, with any number of types!

Fold Expressions (since C++17)

A fold expression expands an expression with *n* parameters as follows:

- Unary Right Fold:

  (E op …) becomes (E_1 op (... op (E_{n-1} op E_n)))

  ({1,2,3,4} + …) becomes (1 + (2 + (3 + 4)))

Fold Expressions (since C++17)

A fold expression expands an expression with *n* parameters as follows:

- Unary Right Fold:

  (E op …) becomes (E_1 op (... op (E_{n-1} op E_n)))

- Unary Left Fold:

  (... op E) becomes (((E_1 op E_2) op …) op E_n)

Fold Expressions (since C++17)

A fold expression expands an expression with *n* parameters as follows:

- Unary Right Fold:
  (E op …) becomes (E_1 op (... op (E_{n-1} op E_n)))
- Unary Left Fold:
  (... op E) becomes (((E_1 op E_2) op …) op E_n)
- Binary Right Fold:
  (E op … op L) becomes (E_1 op (... op (E_{n-1} op (E_n op L))))
- Binary Left Fold:
  (L op … op E) becomes ((((L op E_1) op E_2) op …) op E_n)